VisionEdge UI Document

Prototyping with GUI

VisionEdge: Latency-Optimized Local Perception & TTS Pipeline

Revision 1.0

11 March 2026

Prepared by

Ayan Gattani
23BCT0059

Madhav Juneja
23BCT0111

Adheesh Garg
23BCT0122

Instructor: Dr. Swarnalatha P

Course: Software Engineering Lab **Lab Section:** L23+L24

Table of Contents

1. Abstract ... 3
2. Introduction ... 3
3. Design Objectives for the Prototype ... 3
   3.1. Immediate legibility ... 3
   3.2. One-step operation ... 3
   3.3. Assistive accessibility ... 4
   3.4. Graceful degradation ... 4
   3.5. Validation before integration ... 4
4. Information Architecture ... 4
5. Primary Screen Specifications ... 5
   5.1. Home / Main Screen ... 5
   5.2. Settings Screen ... 7
   5.3. Permissions / Onboarding Screen ... 9
   5.4. Error and Alert States ... 10
   5.5. Session Summary / Debug Screen ... 11
6. State Model and UI Behavior ... 13
7. UI Components Required at High Priority ... 13
   7.1. Primary action button ... 13
   7.2. Status card ... 13
   7.3. Settings controls ... 13
   7.4. Permission prompt component ... 13
   7.5. Error banner or full-screen error layout ... 14
   7.6. Session event list ... 14
8. Mapping the GUI to Test Cases ... 14
   8.1. Home screen rendering ... 14
   8.2. Settings-driven behavior ... 14
   8.3. Error display and retry ... 14
   8.4. TalkBack compatibility and touch target size ... 14
   8.5. Audio confirmation for actions ... 14
9. Recommended Wireframe Content ... 15
   9.1. Home screen wireframe ... 15
   9.2. Settings screen wireframe ... 15
   9.3. Permissions screen wireframe ... 15
   9.4. Error screen wireframe ... 15
   9.5. Debug screen wireframe ... 15
10. Prototype Development Strategy ... 15
   10.1. Stage 1: static layout ... 16
   10.2. Stage 2: dummy data scenarios ... 16
   10.3. Stage 3: local state and persistence ... 16
   10.4. Stage 4: subsystem attachment ... 16
11. Risks and Interface Implications ... 16
12. Conclusion ... 17

1. Abstract

VisionEdge is an assistive mobile application whose interface must remain dependable under both ordinary and failure-prone conditions. The implementation plan recommends a UI-first approach in which navigation, visual states, dummy data, and interaction flows are stabilized before the full camera, perception, and speech pipeline is integrated. The test specification reinforces this by defining observable acceptance criteria for the home screen, settings, onboarding, error recovery, accessibility, and debug visibility.

This document synthesizes those sources into a prototype-oriented UI specification. The result is not merely a list of screens but a design rationale for how a graphical user interface can support a latency-sensitive assistive workflow. The proposed interface favors large touch targets, explicit state signaling, minimal cognitive load, and robust transitions among idle, active, error, and recovery conditions.

2. Introduction

The VisionEdge application is intended to support users through real-time scene understanding and spoken feedback. Although the eventual product depends on camera capture, on-device inference, and offline text-to-speech, the implementation plan makes a clear architectural recommendation: the interface should be prototyped first, exercised with mock data and deterministic state changes, and only later connected to live subsystems. This sequencing reduces early-stage risk and ensures that interaction design proceeds independently of model integration.

From a software engineering perspective, this is a sound decision. A user-facing product of this kind succeeds only when the interface remains understandable under uncertainty. In VisionEdge, uncertainty appears in several forms: permissions may be denied, models may fail to load, no objects may be detected, device capabilities may differ, and audio output may not always be available. A well-structured GUI therefore acts as the most stable contract between the system and its users.

3. Design Objectives for the Prototype

The GUI prototype should satisfy five objectives that collectively ensure the interface is testable, accessible, and coherent before any backend subsystem is attached.

3.1. Immediate legibility

The user must always be able to determine the current system state at a glance. This requires a strong visual hierarchy with a prominent action button, a concise status label, and a small number of simultaneous controls. Ambiguity in state display is a reliability failure for an assistive product.

3.2. One-step operation

The core interaction is starting or stopping assistance. This workflow must be reducible to a single prominent action, with secondary controls such as settings and retry clearly subordinated in the visual hierarchy.

3.3. Assistive accessibility

Because the product targets accessibility-oriented usage, accessible labels, minimum touch-target sizes, strong contrast, and audio confirmation must be treated as first-class design requirements from the very first prototype iteration, not deferred to a later refinement pass.

3.4. Graceful degradation

The interface must remain coherent when permissions are denied, models fail, the camera becomes unavailable, or audio output is impaired. Each such condition should produce a clear, human-readable message and an unambiguous recovery action.

3.5. Validation before integration

The prototype must be fully exercisable using mock data. A prototype that depends only on simulated state can already validate rendering correctness, navigation flow, interaction timing, copy tone, and screen transitions — without requiring any live backend component.

4. Information Architecture

The VisionEdge application does not require deep information hierarchy. The user's primary intent is singular: activate assistance and receive feedback. The remaining screens exist to support configuration, trust-building, error recovery, and developer verification. A shallow, state-driven navigation model is therefore both appropriate and preferable.

The proposed top-level screens are:

- Home / Main Screen,
- Settings Screen,
- Permissions / Onboarding Screen,
- Error State Screen or modal overlay, and
- Session Summary / Debug Screen.

From	**To**
Permissions / Onboarding	Home / Main
Home / Main	Settings
Home / Main	Error / Recovery
Home / Main	Session Summary / Debug
Error / Recovery	Home / Main
Settings	Home / Main

Table 1: Proposed navigation structure for the UI prototype.

Navigation is intentionally flat. Every non-home screen has a single, unambiguous return path to the home screen, and no screen requires more than two taps to reach from the home screen.

5. Primary Screen Specifications

The following subsections specify layout, content, and behavioral expectations for each screen in the prototype.

5.1. Home / Main Screen

The home screen is the operational center of the application. According to the implementation plan, it must include a clear product title, a current system status indicator, the primary start or stop interaction, and an entry point to settings. The test specification further requires that the title, status label, primary button, and settings icon all render correctly and remain visible across device configurations.

A suitable layout is organized into the following regions:

- a top app bar containing the product name and a settings action icon,
- a central status panel showing operational state and supporting detail,
- a dominant start/stop button below the status panel,
- a small readiness indicator strip for camera, speech, and model status, and
- an optional footer showing the most recent narration or event.

The status label must communicate operational modes explicitly. Suitable values include **Idle**, **Starting**, **Listening**, **Active**, **No objects detected**, and **Error**. These labels are essential because the product involves asynchronous initialization stages that are otherwise opaque to the user.

The main button should follow state-driven labeling:

- **Start Assistance** when idle and ready,
- **Stop Assistance** when active,
- **Starting...** (disabled) during initialization, and
- **Retry** (replacing or supplementing the primary action) when an error blocks operation.

A useful typographic convention is to keep the status label short and decisive — **Camera permission needed** — while a secondary line beneath it provides actionable guidance: "Open Settings to grant camera access."

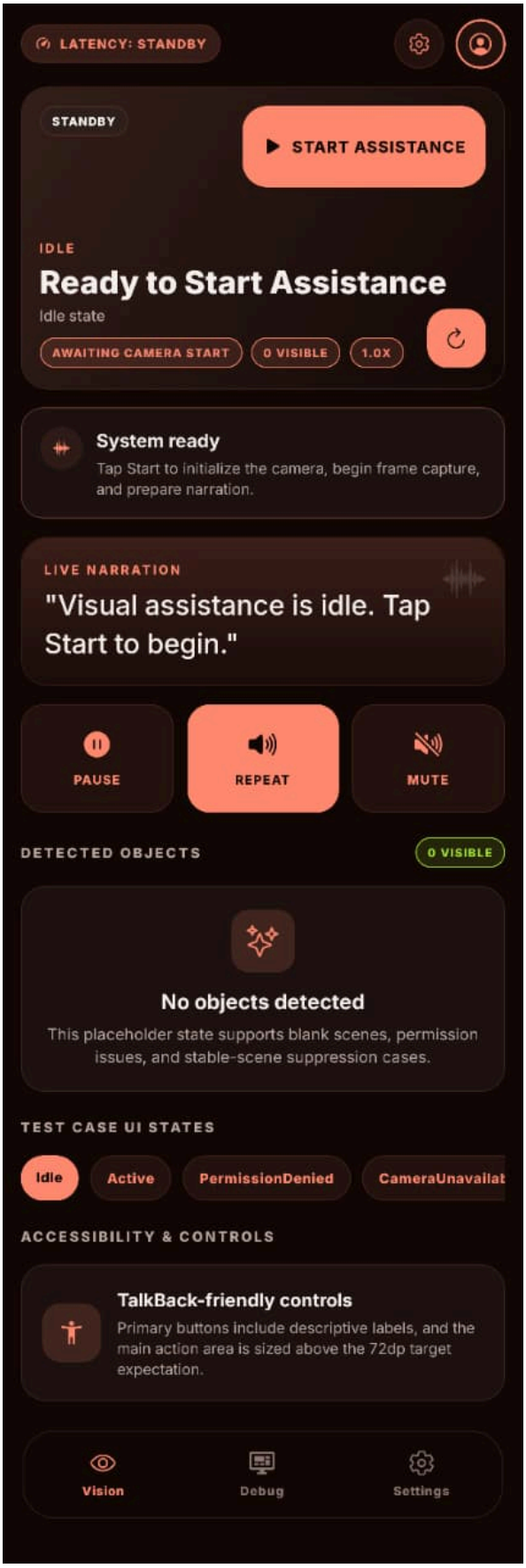

Figure 1: Home screen prototype showing the primary action button and status panel.

5.2. Settings Screen

The settings screen directly influences behavioral expectations tested by the test specification. Speech rate and verbosity level are explicitly listed as acceptance criteria. These controls must therefore be discoverable, clearly labeled, and produce visible feedback immediately upon interaction, even when real TTS is not yet wired.

Recommended sections and their contents:

- **Speech Output** — speech rate slider with numeric readout, volume behavior note, and an audio confirmation toggle for later extension;
- **Description Style** — verbosity selector offering **Minimal**, **Balanced**, and **Detailed** options;
- **Feedback and Accessibility** — haptic feedback toggle, large-label and high-contrast mode placeholders, and explicit content-description notes for assistive technology support;
- **System** — debug mode toggle and reset-to-defaults action.

The settings screen is an ideal early candidate for state wiring. Even before real TTS is available, the interface can respond to value changes immediately and display mock confirmation text such as "Speech rate set to 2.0×", allowing settings-related test cases to pass against the prototype.

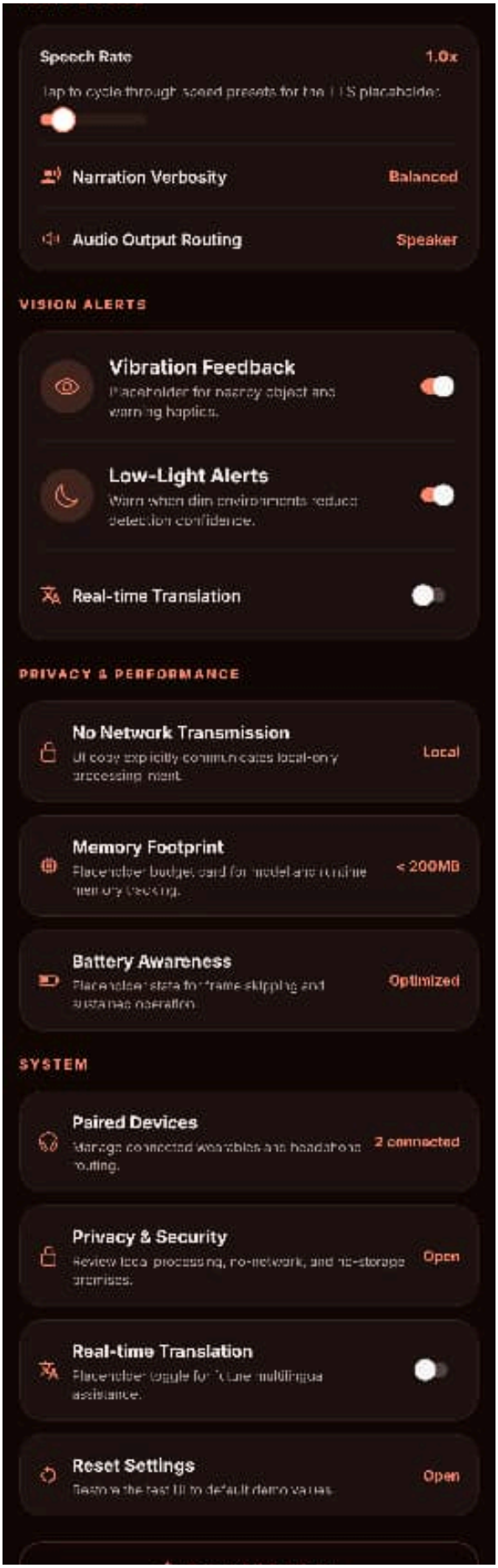

Figure 2: Settings screen showing configurable speech and accessibility controls.

5.3. Permissions / Onboarding Screen

The implementation plan prescribes a dedicated permissions or onboarding surface. For an assistive camera application this screen is mission-critical: it establishes trust, explains camera and audio usage, and initiates the permission request. The user must understand three things before the assistance session begins:

1. why the camera is required,
2. that all processing is local and privacy-preserving, and
3. what kind of audio feedback the application will provide.

The onboarding sequence should remain short. A single-page or three-card presentation is sufficient:

1. purpose of assistance,
2. camera and audio permission explanation, and
3. privacy and offline behavior summary.

The call-to-action must be explicit: **Grant Camera Access** or **Continue**. Copy must be unambiguous; permission denial is a hard blocker for the core feature, not a degraded-mode scenario.

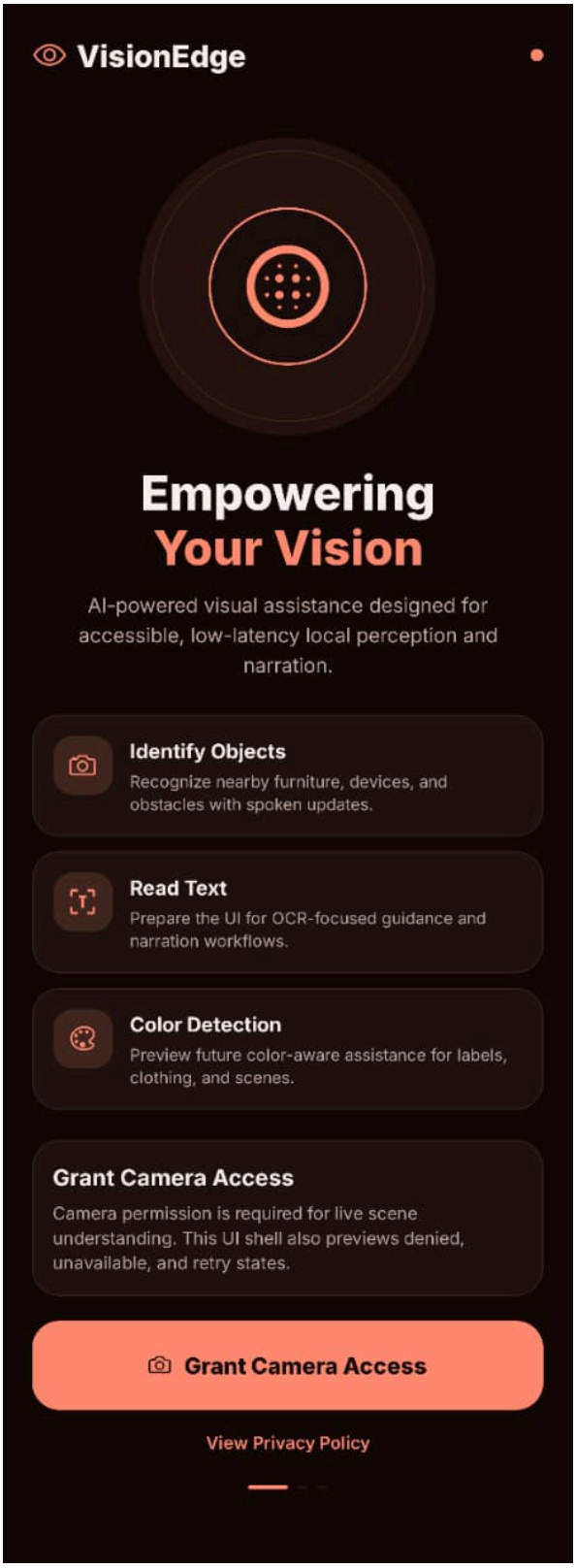

Figure 3: Onboarding and permissions screen presented before assistance begins.

5.4. Error and Alert States

The implementation plan recommends either a dedicated screen or a component-based error overlay. The test cases require that permission failure, unavailable camera access,

and model load failure all be handled gracefully, with clear messaging and a retry action where recovery is possible.

A well-structured error state contains:

- an error icon or visual signifier,
- a short title naming the specific issue,
- one or two lines explaining the consequence for the user,
- a primary recovery action (typically **Retry** or **Open Settings**), and
- an optional secondary action for dismissal or further navigation.

Representative prototype error variants include:

- camera permission denied,
- camera device unavailable,
- audio output unavailable,
- model failed to load,
- no objects detected (informational, inline), and
- low confidence or low-light condition (warning level).

Not all error conditions require full-screen treatment. Transient or informational states such as **No objects detected** may remain inline on the home screen. Blocking failures such as **Model failed to load** deserve a dedicated, full-screen state that prevents the user from interacting with a non-functional system.

5.5. Session Summary / Debug Screen

The implementation plan explicitly calls for a session summary or debug screen. The test specification depends on observable internal behavior — frame counts, initialization timing, memory conditions, and recovery transitions — making a dedicated debug view essential during prototype validation. This screen should be toggled via the debug mode setting and remain invisible to end users in release builds.

A suitable prototype debug screen includes:

- current app state and last state transition,
- camera readiness and frame rate,
- model load status and load time,
- most recent detected objects from dummy data,
- most recent generated description,
- current speech rate and verbosity level,
- a mock latency breakdown, and
- a chronological event log.

This screen also supports validation of non-visual requirements such as frame-skip logic, queue handling, and delegate usage as subsystems are integrated.

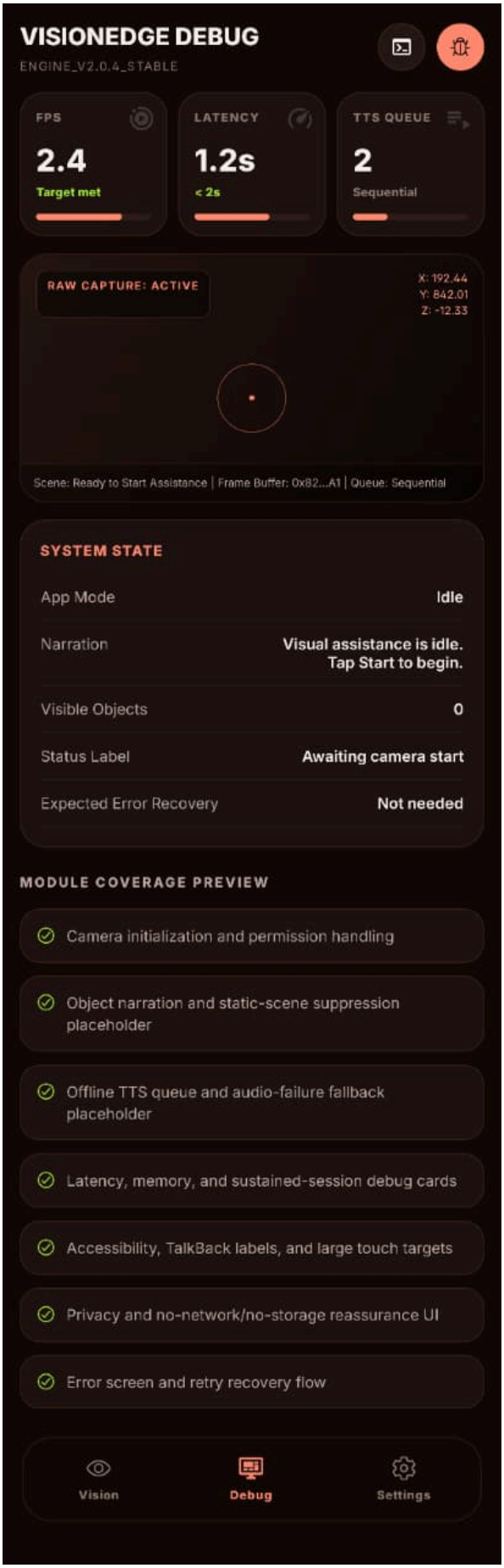

Figure 4: Debug and session summary screen for observing internal state during prototyping.

6. State Model and UI Behavior

The implementation plan recommends a structured state model. For the prototype, the visible GUI can be derived from a finite set of application states:

$$S = \{\text{Idle}, \text{Initializing}, \text{Active}, \text{Warning}, \text{Error}, \text{Recovering}\} \tag{1}$$

Each state determines the set of enabled actions and the wording of all status surfaces. No screen should display content that contradicts the current state, and no interactive control should be available if it cannot produce a meaningful result in the current state.

The valid transition set is as follows:

- Idle → Initializing after the user activates start,
- Initializing → Active if camera and speech resources become available,
- Initializing → Error if a critical dependency fails,
- Active → Warning if non-fatal issues arise such as empty detections,
- Active → Error if the camera or audio pipeline fails mid-session,
- Error → Recovering after the user activates retry,
- Recovering → Idle or Recovering → Active depending on subsystem readiness, and
- Active → Idle after the user activates stop.

This explicit state orientation is important because several test cases evaluate transition correctness rather than static visual appearance alone.

7. UI Components Required at High Priority

The following components must be designed and implemented before any subsystem attachment begins. Their correctness is a precondition for passing the majority of the defined test cases.

7.1. Primary action button

The most important control in the system. It must meet the minimum accessible touch-target specification, visually distinguish active and inactive modes through both color and label, and always communicate the prospective action rather than the current state alone. An active system should show **Stop Assistance**, not **Active**.

7.2. Status card

A prominent panel anchoring the center of the home screen. It must display the current operational state, the most recent relevant event, and any critical blocking message. This component carries most of the user's trust; an ambiguous or stale status card is a usability failure.

7.3. Settings controls

Sliders, segmented controls, toggles, and value readouts must be consistent in visual language and behavior. Their primary testing obligation is that value changes produce observable, immediate feedback even in mock mode.

7.4. Permission prompt component

A reusable component that explains the missing permission and provides a direct action path. Because permission denial is a tested failure mode, this component must appear

both during onboarding and in mid-session recovery flows, with context-appropriate copy in each case.

7.5. Error banner or full-screen error layout

A severity-aware component supporting both inline banner presentation for transient issues and full-screen blocking layout for critical failures. The two visual modes must be visually distinct so that severity is immediately legible.

7.6. Session event list

A chronological list of application events supporting both debug visibility and interaction transparency. Even in mock mode, entries such as "Camera initialized", "Narration queued", and "No objects detected" make the system feel operational and allow testing of non-visual behavioral requirements.

8. Mapping the GUI to Test Cases

The test specification imposes several direct obligations on the prototype interface, each of which maps to a specific design decision.

8.1. Home screen rendering

The prototype must render the title, status label, primary button, and settings affordance in a stable, uncluttered layout. This argues against a visually busy home screen. A minimal layout improves both user clarity and test conformance.

8.2. Settings-driven behavior

The settings screen must visibly affect the application even in mock mode. The interface can simulate speech rate and verbosity changes through preview text, descriptive labels, and example narration strings, producing testable outcomes without a live TTS backend.

8.3. Error display and retry

The error-state design must support both recognition and recovery. A successful retry is not solely a backend concern; it must be legible as a visual transition from a failure state back to an initialized ready state, with the primary button re-enabling and the status label updating accordingly.

8.4. TalkBack compatibility and touch target size

These test requirements directly govern component sizing and labeling strategy. Every interactive control must carry an accurate accessible description. Buttons must not rely on iconography alone. Minimum touch-target dimensions must be enforced in the layout system, not left to individual screen implementations.

8.5. Audio confirmation for actions

Although audio output is a subsystem feature, the interface must acknowledge that user commands have been registered. The prototype supports this through transient status line updates or lightweight confirmation overlays paired with every major interaction, providing a testable observable signal even without live TTS.

9. Recommended Wireframe Content

The following textual wireframes specify the minimum required content for each screen in the first prototype iteration.

9.1. Home screen wireframe

- App bar: product title **VisionEdge**, settings icon (right).
- Status panel: state label (**Idle** / **Starting** / **Active** / **Error**), supporting detail line, readiness indicator strip (camera, model, speech).
- Primary button: **Start Assistance** or **Stop Assistance**, centered, full-width with horizontal padding.
- Footer: most recent narration string or last event entry.

9.2. Settings screen wireframe

- Section **Speech Output**: speech rate slider (range 0.5×–3.0×) with live numeric readout; volume note.
- Section **Description Style**: three-option selector (**Minimal**, **Balanced**, **Detailed**).
- Section **Accessibility**: haptics toggle; large-label placeholder; confirmation cue toggle.
- Section **System**: debug mode toggle; reset-to-defaults action.

9.3. Permissions screen wireframe

- Explanatory card: purpose of camera access, privacy assurance (local processing only), expected audio behavior.
- Permission status row: camera granted / denied indicator.
- Primary action: **Grant Camera Access**.
- Secondary action: **Continue** (available only when permission is already granted).

9.4. Error screen wireframe

- Error icon (severity-coded).
- Error title: concise issue name.
- Detail text: one or two lines explaining user impact.
- Primary action: **Retry** or **Open Settings**.
- Optional secondary action: **Dismiss** or **Go Home**.

9.5. Debug screen wireframe

- State row: current app state and last transition.
- Hardware row: camera state, frame rate.
- Model row: load status, load latency.
- Inference row: last detected objects, last description string.
- Configuration row: active speech rate, verbosity level.
- Event log: timestamped entries, newest first.

10. Prototype Development Strategy

The implementation plan's milestones imply a disciplined rollout sequence. In interface design terms, these milestones correspond to four distinct prototype stages.

10.1. Stage 1: static layout

Build all screens using placeholder text and fixed values. Validate typography, spacing, navigation paths, and control visibility. No state logic is required at this stage; the goal is a complete, navigable shell.

10.2. Stage 2: dummy data scenarios

Introduce scripted state transitions: permission denied, camera ready, empty detection, multiple objects detected, and model load failure. This provides substantive UI exercise and allows the majority of rendering and layout test cases to be executed without any backend dependency.

10.3. Stage 3: local state and persistence

Wire settings and recent-state retention into local storage. The prototype should now behave consistently across sessions and reflect configuration changes immediately. Settings-driven behavioral test cases can be fully addressed at this stage.

10.4. Stage 4: subsystem attachment

Connect permissions, camera indicators, and TTS confirmation signals incrementally. Each integration must preserve the same UI contract validated in mock mode. Regressions against previously passing test cases should be treated as blocking issues.

This staged sequence ensures that the user experience remains coherent as technical dependencies grow more complex, and that every integration step can be evaluated against an already-validated baseline.

11. Risks and Interface Implications

The implementation plan identifies several engineering areas of elevated difficulty. Each has a direct consequence for interface behavior.

Initialization latency. Real-time mobile inference may involve nontrivial startup time for model loading and camera warm-up. The interface must communicate progress during this period rather than appearing unresponsive. The **Initializing** state and a visible progress signal address this directly.

Inference confidence variability. Detection confidence will fluctuate with lighting, camera angle, and scene complexity. The interface should handle low confidence through warning-level messaging rather than silence, avoiding the impression that the system has failed when it is in fact operating within expected parameters.

Device-level latency variation. Frame rates and inference times will differ across hardware configurations. A debug surface allows developers to observe this variation during testing without burdening the home screen with technical metrics that would be meaningless to end users.

Offline operation and privacy. The test specification prohibits network transmission of image data. The interface should reinforce this constraint by stating that all processing occurs locally, both on the permissions screen and in the settings summary. This framing builds user trust and aligns the UI copy with the system's actual behavior.

12. Conclusion

The VisionEdge GUI should be treated as the first operational product artifact, not as a visual shell waiting for backend completion. The implementation plan strongly supports this sequencing, and the test specification confirms that many acceptance criteria are interface-visible long before full perception and speech systems are production-ready.

A successful prototype requires a stable home screen that communicates state unambiguously, a settings page that produces observable behavioral changes, a short and trust-building onboarding flow, explicit error handling with clear recovery paths, and a debug surface that makes internal system behavior visible during development. Across all screens, the design must privilege clarity, accessibility, recoverability, and state transparency.

If these qualities are established in the earliest prototype stages, the subsequent integration of camera access, on-device inference, and offline speech synthesis becomes substantially less risky. The interface contract is already validated; each subsystem integration needs only to satisfy it.